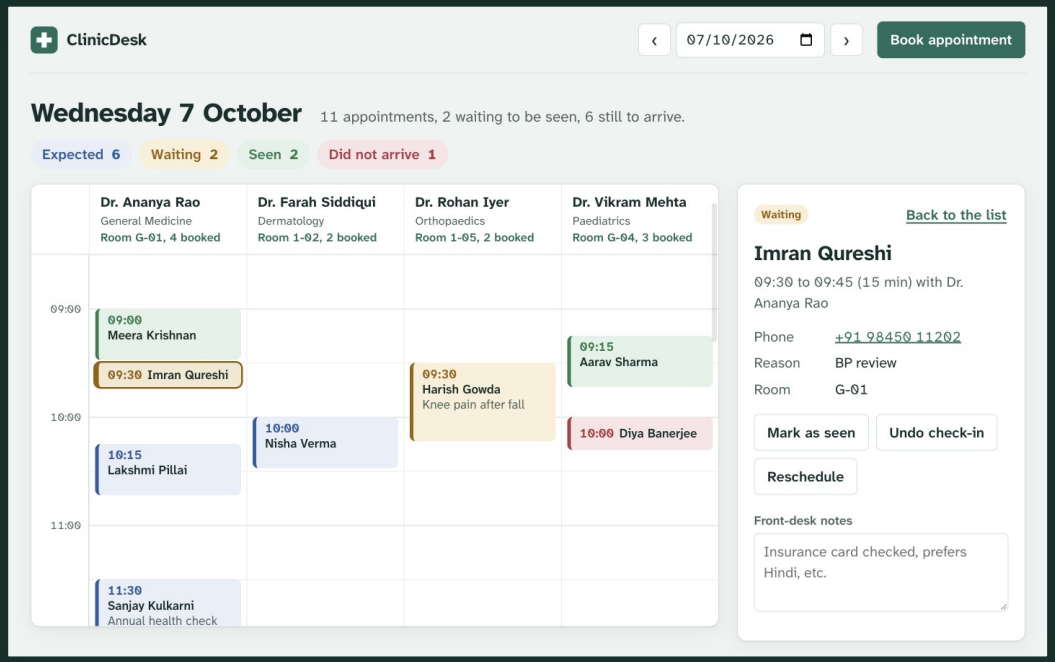
ClinicDesk

From commit to cluster: shipping a clinic front-desk app with DevSecOps

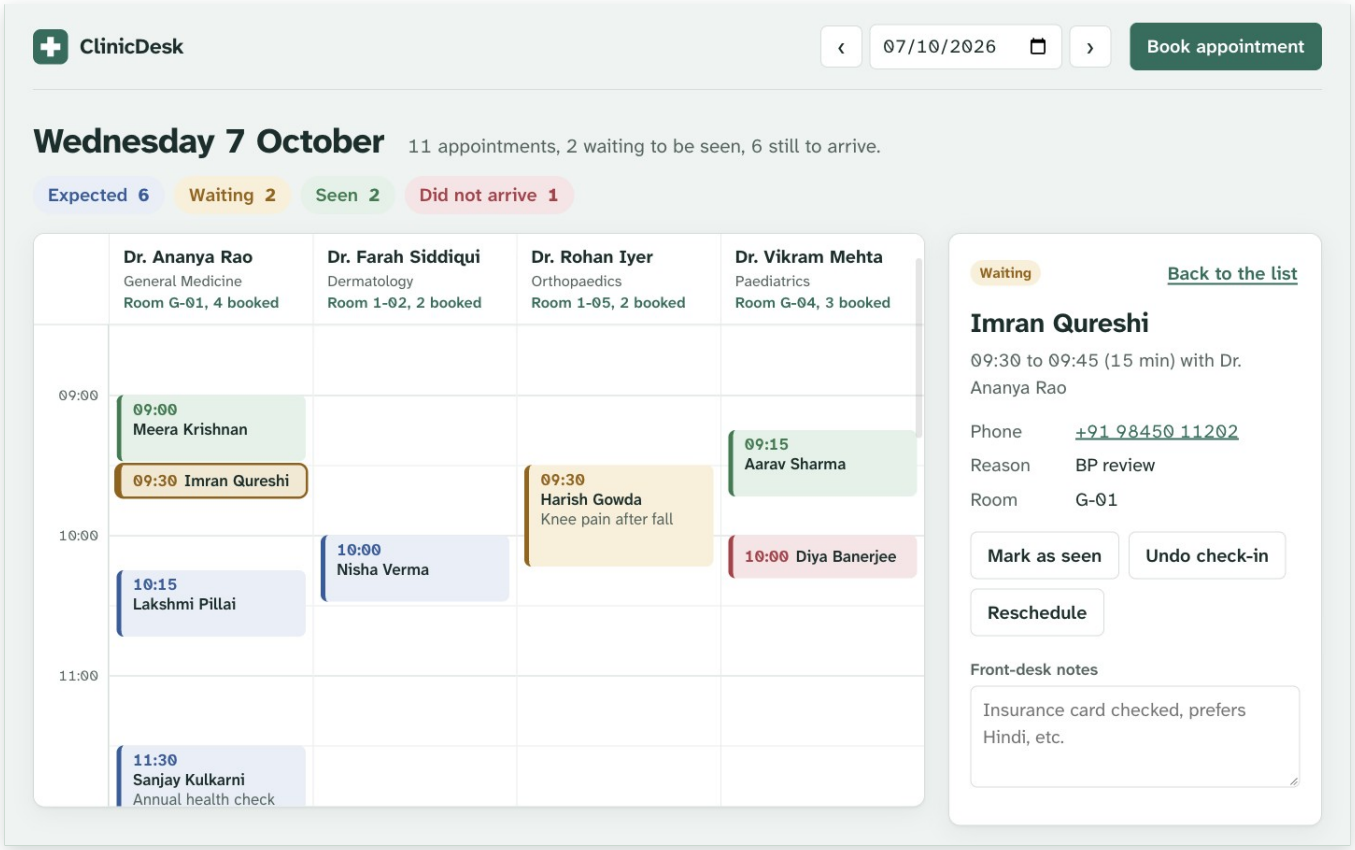
Session 21 final capstone · SST DevOps & Cloud [SWE]

Saptak Banerjee · October 2026



FastAPI · React · PostgreSQL · Docker · GitHub Actions · Trivy · Terraform · EKS · Helm · Argo CD · Prometheus · Grafana

ClinicDesk runs a clinic's whole day on one board



Real UI served by nginx in front of the FastAPI backend

Day board
One column per doctor on a time grid; colour shows each visit's state.

Safe booking
New or returning patients. The API returns 409 when a doctor would be double-booked.

Visit flow
Check in, mark seen or did-not-arrive, reschedule, cancel, add notes.

Daily stats
Totals per state, per-doctor load and the next 7 days of bookings.

The stack, layer by layer



Frontend

React 19 + Vite 8, served by nginx-unprivileged on :8080



Backend

FastAPI, Uvicorn, Pydantic; /health, /ready, /metrics



Data

PostgreSQL 18, SQLAlchemy 2.1, Alembic migrations



Testing

pytest 9.1 with SQLite in memory, never the real DB



Containers

Multi-stage Alpine images, non-root, Docker Compose



CI/CD + security

GitHub Actions, Trivy, Bandit, Semgrep, pip-audit, gitleaks



Infrastructure

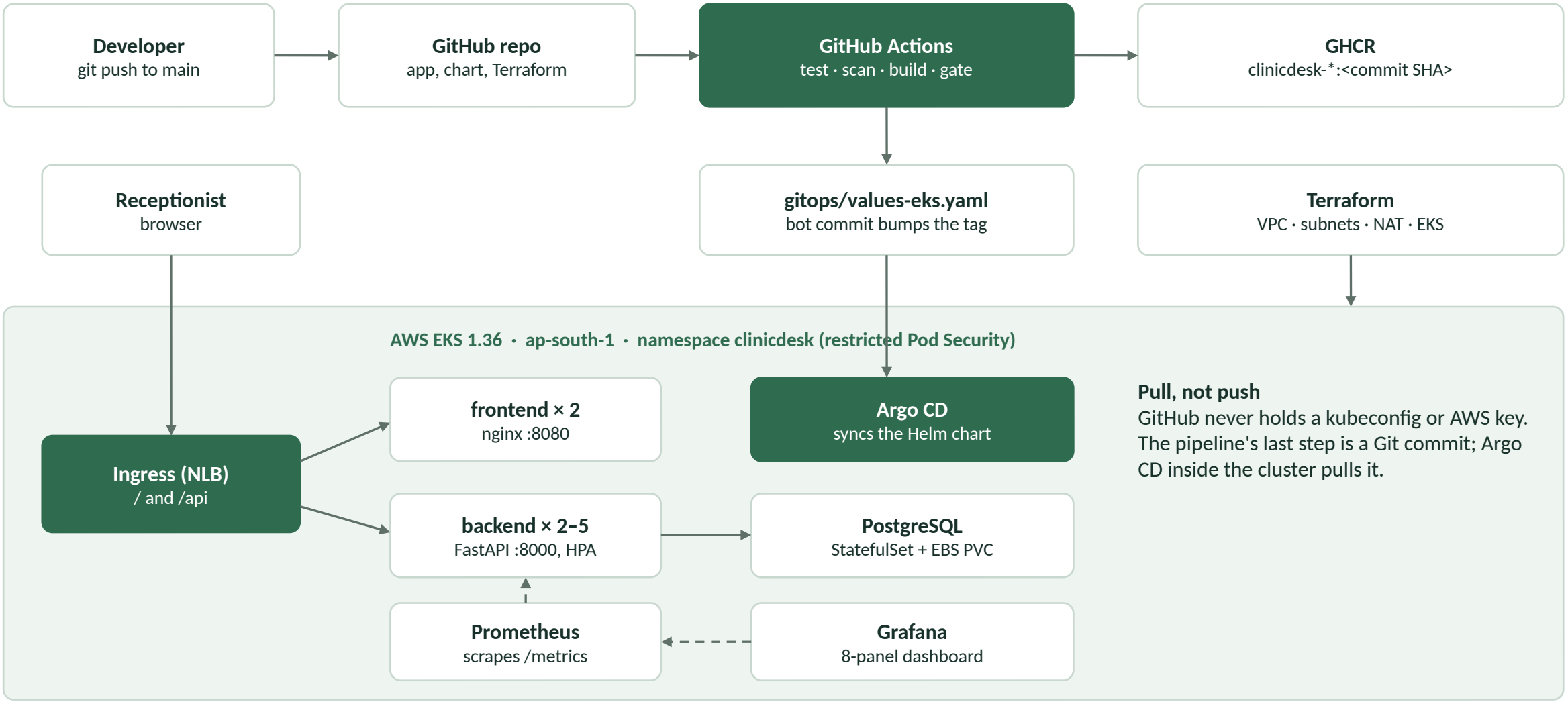
Terraform: VPC, NAT, EKS 1.36 in ap-south-1



Platform

Helm, ingress-nginx, Argo CD, Prometheus, Grafana

Architecture: CI writes to Git, the cluster pulls



Pull, not push
GitHub never holds a kubeconfig or AWS key. The pipeline's last step is a Git commit; Argo CD inside the cluster pulls it.

Application and tests: 23 checks gate every build

23

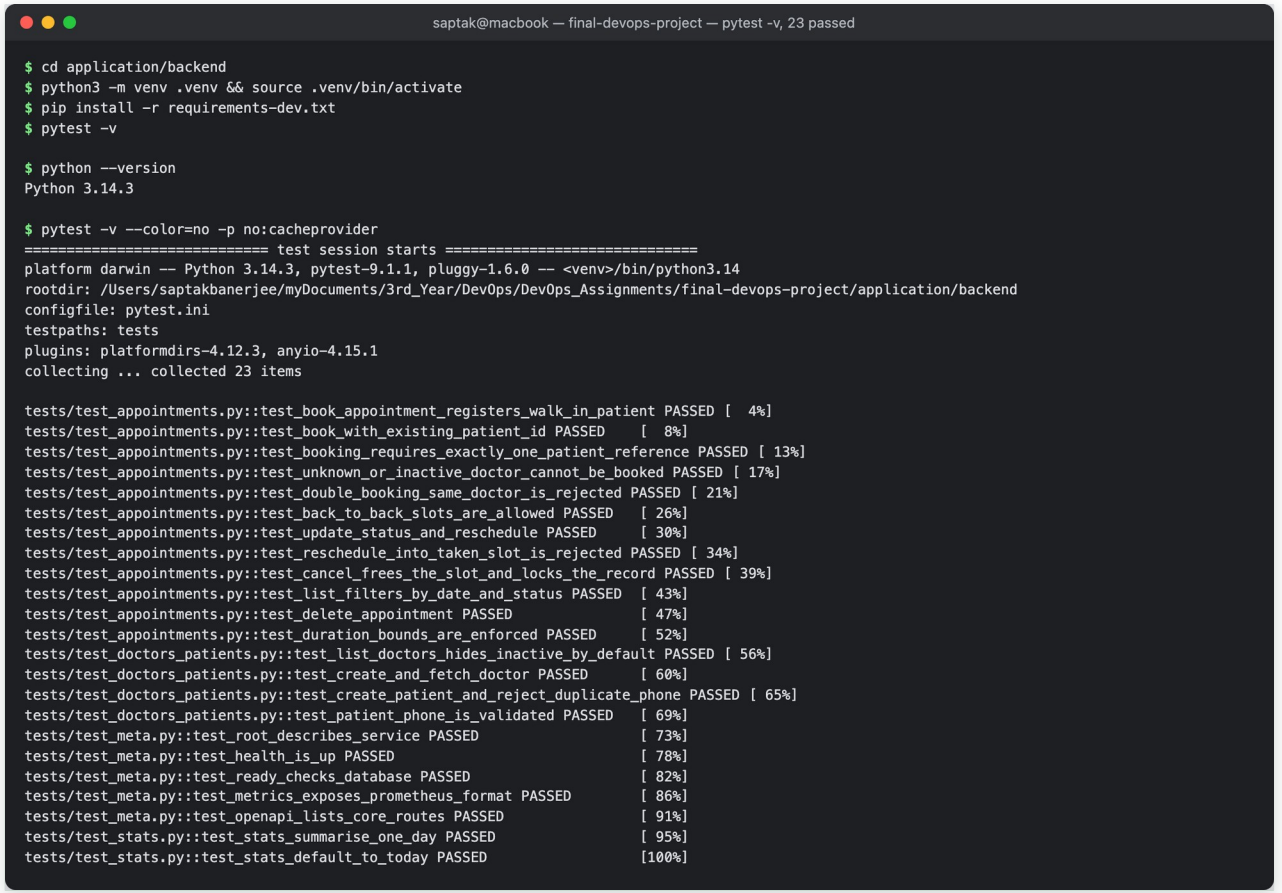
pytest tests pass in 0.64 s, on SQLite in memory, never the real database

3

tables (doctors, patients, appointments) managed by 2 Alembic migrations

409

returned when a booking would clash with a doctor's existing slot



REST API: GET / POST / PUT / DELETE on appointments, plus doctors, patients, stats, /health, /ready, /metrics

Containers: small, non-root, zero HIGH or CRITICAL CVEs



Backend image

- Multi-stage on python:3.14-alpine
- Runs as uid 10001, read-only root filesystem
- Migrates with Alembic, then starts Uvicorn



Frontend image

- Stage 1: node:24-alpine runs vite build
- Stage 2: nginx-unprivileged serves dist/
- Proxies /api to the backend



docker compose up --build

- frontend on localhost:3000
- backend on :8000 with health checks
- PostgreSQL 18 with a named volume

44 → 0

HIGH CVEs in the backend image after moving from a Debian-slim base, where none had a fix, to Alpine. Ignoring unfixed CVEs would have turned the scan green while shipping all 44.

CI/CD: ten jobs from a push to a promoted release

1 Test: pytest + vite build	**2** SAST: Bandit, Semgrep	**3** SCA + IaC: pip-audit, npm audit, Trivy	**4** Secrets: gitleaks	**5** Docker build (both images)
6 Image scan: Trivy	**7** **Security gate**	**8** Push to GHCR as :<SHA>	**9** Helm install on kind	**10** GitOps promote

A real run on GitHub Actions, commit dbbdd49, 4 min 26 s end to end



Triggers: every push to main that touches the project, plus pull requests. **Tags:** ghcr.io/saptak-cyber/clinicdesk-{backend,frontend}:<commit SHA>, never latest.

DevSecOps: nine reports, one gate that fails closed

SAST Bandit, Semgrep

SCA pip-audit, npm audit, Trivy fs

IaC Trivy config on K8s, Helm, Terraform

Secrets gitleaks, full history

Images Trivy on both images

Security gate
security_gate.py applies gate-policy.toml to every report. A missing report fails.

What Trivy found

Trivy scanned both images' OS packages and their Python and npm libraries. The final run had 0 HIGH and 0 CRITICAL, so the gate passed.

Findings fixed on the way: 44 unfixable HIGH CVEs in Debian-slim (removed by moving to Alpine), Bandit B608 f-string SQL in a migration, and a writable root filesystem on PostgreSQL.

Gate verdict on the live-demo run: all nine checks PASS, SECURITY GATE: PASSED. Any HIGH or CRITICAL finding fails the build before an image is pushed.

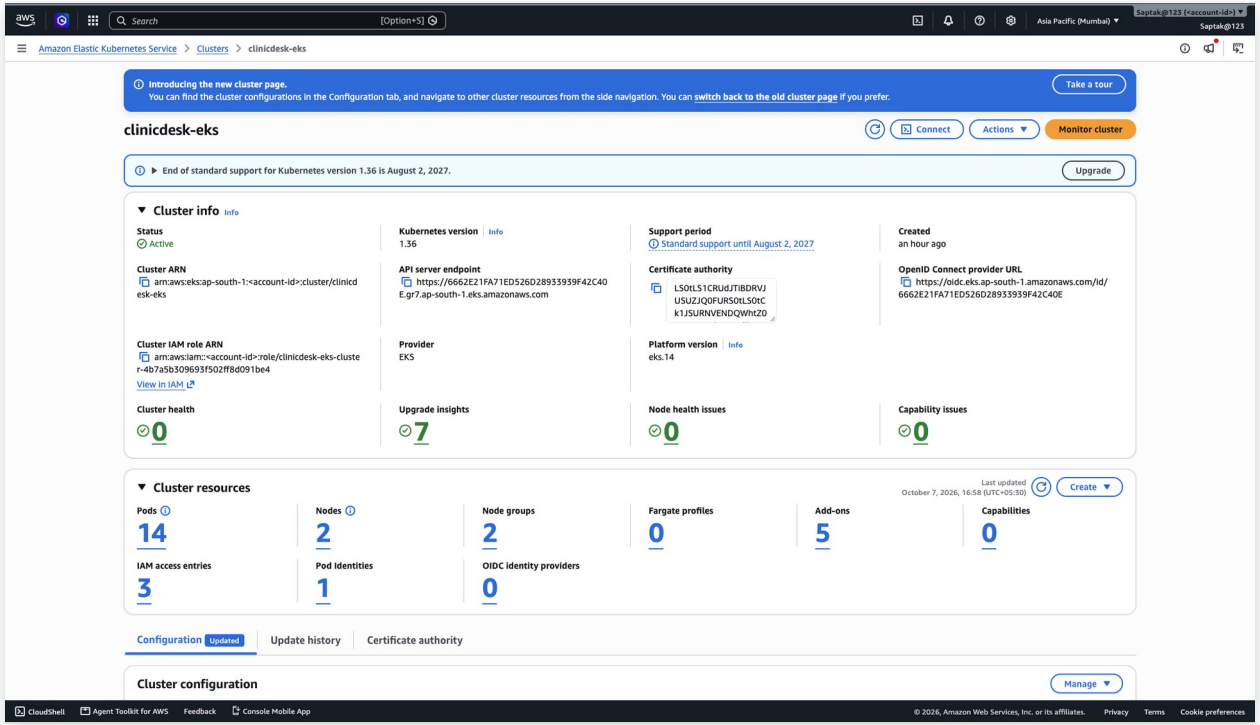
Terraform builds the AWS foundation in one apply

61
resources created

14 min
apply, 9 of it the EKS control plane

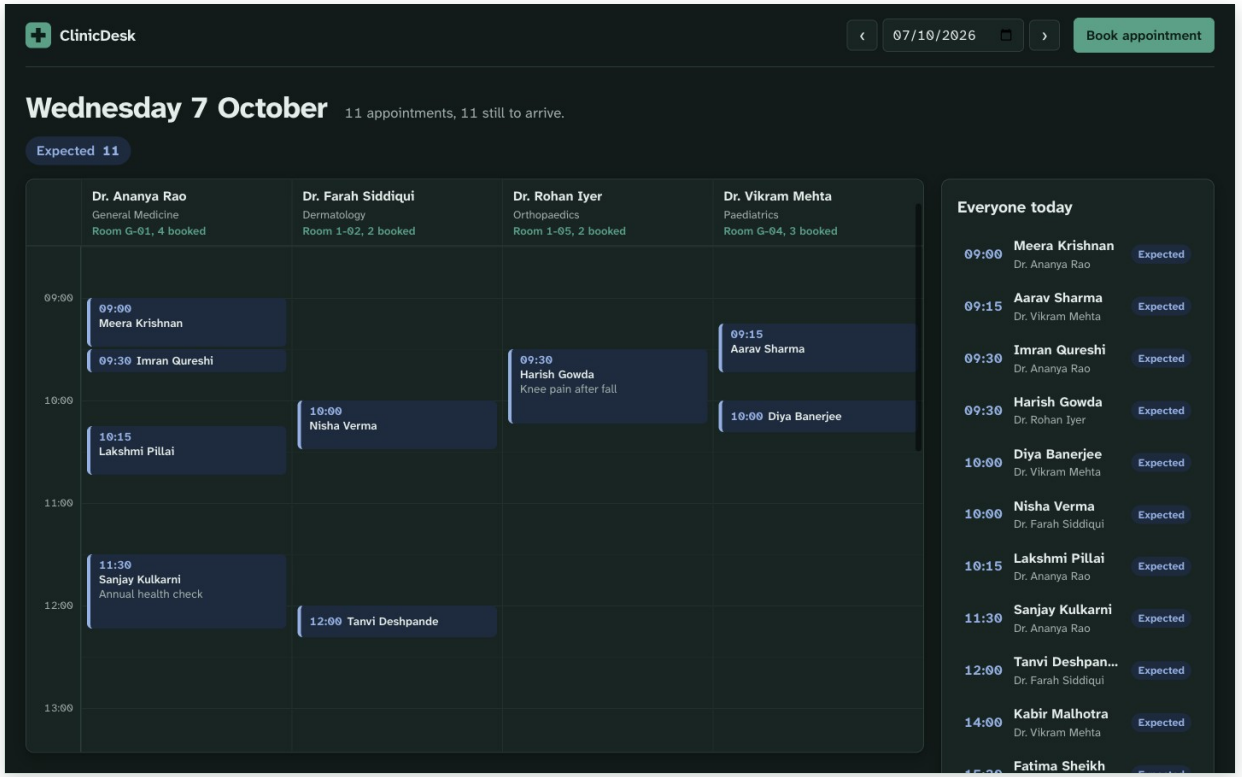
0
resources left after destroy

- VPC 10.42.0.0/16: 2 public + 2 private subnets across ap-south-1a and 1b, one NAT gateway
- EKS 1.36, managed node group of t3.medium (min 1, desired 2, max 3) in private subnets
- Add-ons: VPC CNI, CoreDNS, kube-proxy, metrics-server, EBS CSI via Pod Identity
- Only terraform.tfvars.example is committed; no credentials in Git



clinicdesk-eks Active in Asia Pacific (Mumbai): 2 nodes, 5 add-ons

Kubernetes and Helm: one chart for dev, prod and EKS



The app on EKS, reached through the ingress-nginx NLB

Namespace
clinicdesk with restricted Pod Security

Deployments
frontend and backend, 2 replicas each; backend HPA 2–5 at 60% CPU

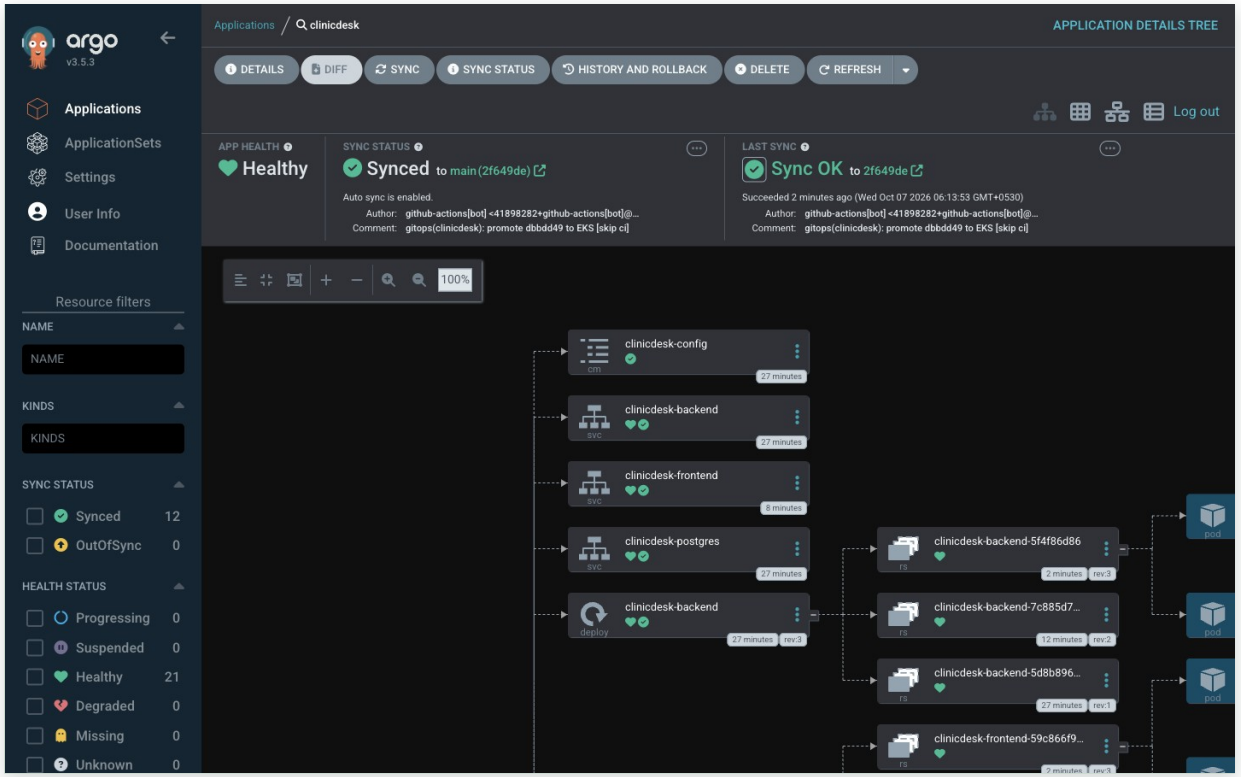
Services + Ingress
ClusterIP Services; Ingress sends / to frontend, /api to backend

PostgreSQL
StatefulSet with a 5 GiB gp3 PVC on EBS

Helm values
values-dev, values-prod, gitops/values-eks

GitOps: Argo CD keeps the cluster equal to Git

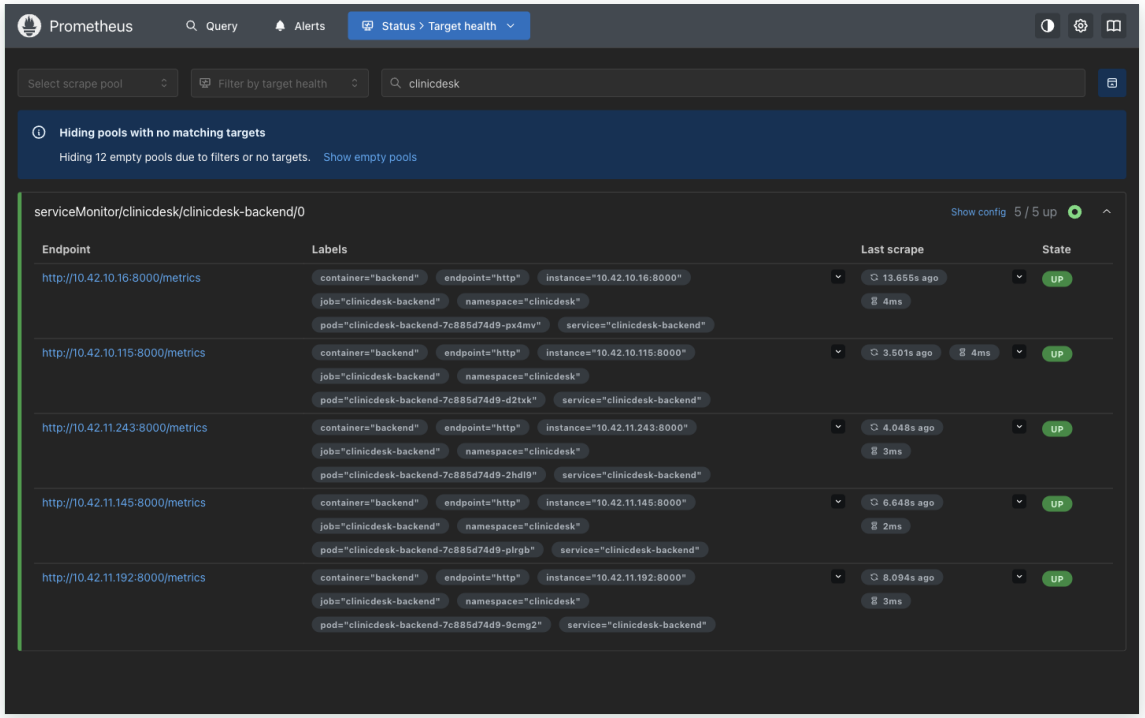
1. **Commit**
   ClinicDesk 1.1.0 adds opening hours to the footer
2. **Pipeline**
   Tests, scans, gate, push dbbdd49 images to GHCR
3. **Promote**
   Bot commit sets both tags in values-eks.yaml [skip ci]
4. **Sync**
   Argo CD polls every 60 s and rolls out the new SHA



Argo CD: clinicdesk Synced and Healthy at the bot's promotion commit

6 s for self-heal to undo a manual ConfigMap edit and a deleted Service

Observability: every pod scraped, every request graphed



Prometheus: a ServiceMonitor scrapes /metrics every 15 s; 5 of 5 backend pods UP during load.



Grafana: 8 panels (request rate, error ratio, p50/p95 latency, HPA replicas, CPU, memory) plus 3 alert rules.

Key learnings

01

No fixable vulnerabilities is not no vulnerabilities

Changing the base image removed 44 HIGH CVEs that ignore-unfixed would have hidden.

02

Pods, not cores, filled the cluster

With the VPC CNI a t3.medium holds 17 pods; an autoscale plus rollout hit that limit.

03

Liveness and readiness answer different questions

Database down made pods unready without a single restart, and they recovered by themselves.

04

GitOps changes how you fix things

Self-heal undid manual changes in seconds, so the fix has to go into Git.

05

The API server's defaults are part of the diff

A StatefulSet stayed OutOfSync until the chart spelled out the defaulted fields.

06

Pull-based delivery removes a secret

GitHub never held a kubeconfig or AWS key; deployment is a Git commit.

ClinicDesk

Next: the live walkthrough

1. Codebase and folder structure
2. CI/CD: push and watch the pipeline
3. Terraform plan and the AWS console
4. Argo CD: synced app and GitOps flow
5. Kubernetes: pods, services, ingress
6. Prometheus targets UP
7. Grafana live dashboard

Repository
github.com/Saptak-cyber/DevOps_Assignments/tree/main/final-devops-project

Full technical write-up
final-devops-project/README.md